

Движение, Ownership и стрельба

Карта занятия:

- ① **NetworkTransform** — Jitter Buffer, жанровые рецепты, NetworkRigidbody
- ② **Ownership** — ChangeOwnership, жизненный цикл, топ-3 ошибки
- ③ **Камера** + серверный спавн объектов
- ④ **Стрельба** — валидация RPC, полный цикл от ввода до урона

Jitter Buffer — почему движение плавное

① NetworkTransform

Проблема: пакеты приходят каждые 50 мс, но экран обновляется каждые 16 мс. Без буфера — объект стоит → прыгает → стоит.

Решение — Jitter Buffer:

- ▶ Держим буфер из 2-3 пакетов вперёд
- ▶ Намеренно задерживаем отображение на 2-3 тика
- ▶ Рендер всегда читает из буфера → никогда не «заикается»

Аналогия: YouTube-буфер

Загрузка идёт чуть вперёд воспроизведения → при небольшом лаге сети видео не останавливается

**Цена: +50-100 мс задержки
отображения**

Результат: плавное движение даже при

**NetworkTransform включает это по
умолчанию**

Настройка через Inspector: Interpolation = On

Сеть: [P1] — [P2] — [P3] — [P4] (каждые 50мс)

Рендер: [f][f][f][f][f][f][f][f][f][f] (каждые 16мс) → Lerp(P1→P2)

Jitter Buffer — аналогия: YouTube

① NetworkTransform

YouTube буфер

- Видео загружается чуть вперёд воспроизведения
- При небольшом лаге — видео не останавливается
- Цена: небольшая задержка начала

Jitter Buffer в мультиплеере

- Позиции других игроков приходят каждые 50 мс
- Держим 2-3 пакета в буфере, показываем чуть позади
- Цена: +50-100 мс. Результат: никогда не заикается

NetworkTransform включает Jitter Buffer автоматически

Inspector: Interpolation = On

NetworkTransform vs NetworkRigidbody

① NetworkTransform

NetworkTransform

Синхронизирует **позицию**

- ☒ CharacterController
- ☒ Платформа, дверь (редкие апдейты)
- ☒ ❌ С Rigidbody → физика расходится

NetworkRigidbody

Синхронизирует **velocity**

- ☒ Мяч, машина с физикой
- ☒ Снаряд с Rigidbody

Шпаргалка

- Персонаж с CharacterController → **NetworkTransform**
- Физический объект (мяч, машина, снаряд) → **NetworkRigidbody**
- Статический (дверь, лифт) → **NetworkTransform (обновления редкие)**

Ownership: ключевые концепции

② Ownership

IsOwner — recap

```
void Update()
{
    if (!IsOwner) return;
    // только хозяин обрабатывает ввод
    HandleMovement();
}
```

Кому принадлежат объекты:

- Player_0 → Client 0
- Player_1 → Client 1
- NPC, предметы → Server

ChangeOwnership — новое

- Игрок подобрал предмет → предмет его
- Сел в машину → машина его
- Умер → зритель-камера его

Только сервер вызывает!

```
// Только на сервере!
networkObject.ChangeOwnership(newClientId);
```

⚠ Важно: Player Prefab автоматически принадлежит подключившемуся клиенту

Все остальные объекты (NPC, предметы) — принадлежат серверу по умолчанию

IsOwner — ресап за 1 минуту

② Ownership

Повтор из Лекции 1 —
кратко

Каждый NetworkObject принадлежит одному клиенту (OwnerClientId)

Player Prefab → автоматически Client. NPC, предметы → по умолчанию Server.

// Единственная строка в Update:

```
void Update() {  
    if (!IsOwner) return;  
    // чужие — пропускаем  
    HandleMovement();  
}
```

Флаги состояния

- **IsOwner** — я хозяин объекта
- **IsServer** — я сервер (и Host)
- **IsHost** — клиент+сервер вместе
- **IsClient** — любой клиент

Без if (!IsOwner) return: все игроки управляют всеми объектами → персонажи сходят с ума

ChangeOwnership — передача владения

② Ownership

NetworkObject.ChangeOwnership(ulong newOwnerId) — только на сервере!

Кейс 1: Подбор оружия / предмета

- Оружие на земле — владелец: Server
- Игрок подошёл → PickupServerRpc() → ChangeOwnership(player.OwnerClientId)

Кейс 2: Турель / транспорт

- Игрок садится в турель → получает управление
- Выходит → ChangeOwnership(ServerClientId)

```
// ServerRpc вызванный клиентом → сервер выполняет
void PickupServerRpc(ServerRpcParams rpcParams = default)
{
    ulong callerId = rpcParams.Receive.SenderClientId;
    GetComponent<NetworkObject>().ChangeOwnership(callerId);
}
```

Жизненный цикл: Spawn → Despawn

② Ownership

Instantiate

`GameObject.Instantiate(prefab)`



Spawn()

`NetworkObject.Spawn() / SpawnWithOwnership()`



OnNetworkSpawn()

Инициализация: камера, UI, подписки. IsOwner корректный!



Update / Runtime

Логика игры, RPC, `NetworkVariable.OnValueChanged`



OnNetworkDespawn()

Чистка: отписки, UI off. Вызывается до Destroy!



Despawn(true)

Удаление у всех клиентов

Start() vs OnNetworkSpawn() — порядок вызовов

② Ownership

Что происходит при создании сетевого объекта:

Awake()

Объект создан, сети нет, IsOwner = false



Start()

ОПАСНО! IsOwner = false. Сеть ещё не готова.



NetworkObject.Spawn()

Объект зарегистрирован в сети, OwnerClientId задан



OnNetworkSpawn()

IsOwner ВЕРНЫЙ. Камера, UI, подписки — здесь!

Start() вызывается ДО **Spawn()**. Для сетевой инициализации ВСЕГДА используйте **OnNetworkSpawn()**

Аналогично: **OnDestroy()** — для чистки. **OnNetworkDespawn()** — для сетевой чистки подписок.

Топ-3 ошибки — До / После

② Ownership

1. NetworkVariable в Start()

❌ До:
`start.OnValueChanged +=...`

✅ После:
`OnNetworkSpawn.OnValueChanged +=...`

2. Destroy() вместо Despawn()

❌ До:
`Destroy(gameObject);`

✅ После:
`netObj.Despawn(true);`

3. Нет проверки IsOwner в Update

❌ До:
`void Update() { Move(); }`

✅ После:
`if(!IsOwner)return; Move();`

Камера: проблема одной Main Camera

③ Камера

Проблема:

- На сцене одна Main Camera
- Два игрока заспавнились — камера прыгает на последнего
- Camera.main возвращает непредсказуемый результат

Почему нельзя добавить камеру child к префабу игрока?

Ответ:

- При 2 игроках = 2 копии камеры → Unity не знает главную
- **Решение: одна камера на сцене, привязывается через IsOwner**

Схема:

Player_0 (IsOwner=true) → Camera.main следит за ним | Player_1 (IsOwner=false) → не трогает камеру

Одна камера на всех — следит только за локальным хозяином

Камера: паттерн IsOwner

③ Камера

Паттерн: одна камера на сцене → в `OnNetworkSpawn` привязывается к локальному игроку

```
public override void OnNetworkSpawn() {  
    // Только локальный игрок управляет камерой  
    if (!IsOwner) return;  
  
    Camera cam = Camera.main;  
    cam.transform.SetParent(cameraFollowPoint);  
    cam.transform.localPosition = Vector3.zero;  
    cam.transform.localRotation = Quaternion.identity;  
}
```

`cameraFollowPoint` — пустой Transform на теле/голове персонажа. Нет смещений, нет дрожания.

Камера: FPS / TPS / Top-down

③ Камера

FPS

- Камера = глаза игрока
- Крепим к Head Transform
- localPos = (0, 0, 0)
- Курсор заблокирован

TPS

- Камера = вид сзади/сбоку
- Крепим к CameraRig
- localPos = (0, 2, -5)
- Orbit вокруг игрока

Top-down

- Вид сверху, фиксированный
- Следует за игроком по X/Z
- localPos = (0, 20, 0)
- Курсор свободен

Принцип общий: за `if (!IsOwner) return;` — камера привязывается только к локальному персонажу.

Cursor.lockState — только для локального игрока

③ Камера

Проблема: без IsOwner проверки блокировка курсора применяется ко всем игрокам на хосте → два игрока не могут двигать мышью!

✗ До (блокирует всех)

```
void OnNetworkSpawn() {  
    Cursor.lockState = CursorLockMode.Locked;  
}
```

✓ После (только локальный)

```
void OnNetworkSpawn() {  
    if (!IsOwner) return;  
    Cursor.lockState = CursorLockMode.Locked;  
}
```

Аналогично с Cursor.visible: любые UI-эффекты, связанные с мышью, должны быть за проверкой IsOwner

Правило: всё связанное с локальным вводом, камерой, курсором — за if (!IsOwner) return;

Шаги сетевого спавна объекта

④ Серверный спавн

Только сервер спавнит `NetworkObject` — клиент никогда!

1

```
GameObject obj = Instantiate(prefab, pos, rot); // локальный объект
```

2

```
NetworkObject netObj = obj.GetComponent<NetworkObject>();
```

3

```
netObj.Spawn(); // принадлежит серверу | или SpawnWithOwnership(clientId) — клиенту
```

Instantiate без Spawn: объект создастся только на сервере. Клиенты не увидят его.

```
// Паттерн: клиент просит через RPC, сервер спавнит  
[ServerRpc] void SpawnObjectServerRpc() { ... netObj.Spawn(); }
```

Instantiate vs Spawn — два разных уровня

④ Серверный спавн

Instantiate

- Создаёт GameObject локально
- Только на текущей машине
- Нет NetworkObject — не синхронизируется
- **Использование:** спецэффекты, UI-элементы, звуки

V
S

Spawn

- Регистрирует объект в сети
- Реплицируется ко всем клиентам
- Требуется NetworkObject на объекте
- **Использование:** игровые объекты, снаряды, игроки

Правило: сначала Instantiate, потом Spawn. Только на сервере!

SpawnWithOwnership — снаряд принадлежит стрелку

④ Серверный спавн

Два варианта спавна:

- `networkObject.Spawn()` — принадлежит серверу
- `networkObject.SpawnWithOwnership(clientId)` — принадлежит клиенту

```
// ServerRpc — спавн снаряда с владельцем-стрелком
var obj = Instantiate(projectilePrefab, muzzle.position, Quaternion.LookRotation(dir));
obj.GetComponent<NetworkObject>()
    .SpawnWithOwnership(OwnerClientId); // OwnerClientId = ID стрелка
```

Зачем?

- ✓ В `OnTriggerEnter` снаряда: `OwnerClientId` — ID стрелка. Очки идут правильному игроку.
- ✓ Не нужно передавать `shooterId` отдельно — он уже в объекте.

Despawn — типичная ошибка

④ Серверный спавн

Баг: **Destroy()** без **Despawn()** — объект пропадает у сервера, «труп» остаётся у КЛИЕНТОВ

✗ До

```
void OnTriggerEnter(Collider c) {  
    Destroy(gameObject);  
}
```

✓ После

```
void OnTriggerEnter(Collider c) {  
    if (!IsServer) return;  
    GetComponent<NetworkObject>().Despawn  
    (true);  
}
```

Despawn(destroy: true) — убирает у всех, вызывает OnNetworkDespawn(), уничтожает GameObject

Despawn(destroy: false) — убирает из сети, GameObject остаётся (для пула объектов)

Только сервер вызывает **Despawn()**!

RPC валидация — вспоминаем принцип

⑤ Безопасность RPC

Повтор из Лекции 1 —
кратко

Клиент отправляет НАМЕРЕНИЕ
Сервер принимает РЕШЕНИЕ

✗ Клиент доверенный (опасно)

- Клиент: «Я убил Player_2!»
- Сервер: «Ок, -100 HP»
- Любой читерит!

✓ Клиент недоверенный (правильно)

- Клиент: «Я нажал стрелять в dir X»
- Сервер: проверяет, считает попадание
- Читерство невозможно

Клиент — всегда подозреваемый. Никогда не доверяй ему критические решения.

4 примера читов без валидации

⑤ Безопасность RPC

1. Телепортация

`MoveToServerRpc(Vector3 pos)`

Клиент телепортирует себя куда угодно

2. Бесконечные патроны

`ShootServerRpc()` без cooldown

Флуд снарядами без ограничений

3. Урон чужому игроку

`DealDamageServerRpc(targetId, 9999)`

Мгновенный килл любого игрока

4. Стрельба чужим именем

`ShootServerRpc(shooterId: enemyId)`

Очки начисляются не тому игроку

Как закрыть все эти дыры?

- 1. Никогда не принимай `shooterId` от клиента — используй `OwnerClientId` из RPC
- 2. Проверяй расстояние, боеприпасы, cooldown, состояние объекта на сервере

Следующий слайд — как выглядит правильная валидация

ShootServerRpc — 4 guard-клоза

⑤ Безопасность RPC

```
[ServerRpc(RequireOwnership = true)]  
void ShootServerRpc(Vector3 direction, ServerRpcParams rpcParams = default)  
{  
    // 1. Только владелец объекта может стрелять  
    if (rpcParams.Receive.SenderClientId != OwnerClientId) return;  
  
    // 2. Игрок жив и может стрелять  
    if (!IsAlive || ammo <= 0) return;  
  
    // 3. Cooldown ещё не истёк  
    if (Time.time < lastShotTime + cooldown) return;  
  
    // 4. Направление выстрела физически возможно  
    if (direction.magnitude < 0.1f) return;  
  
    // Всё ок — спавним снаряд  
    lastShotTime = Time.time;  
    ammo--;  
    SpawnProjectile(direction);  
}
```

Полный цикл стрельбы

⑥ Стрельба

Клиент →

Сервер

Input.GetKeyDown
(Client)



ShootServerRpc
(dir)



4 Guard-клоза
✓ Owner ✓ Alive ✓ CD ✓ Dir



SpawnWithOwnership
(снаряд)



PlaySoundClientRpc
(всем клиентам)



OnTriggerEnter
(сервер)

Despawn
снаряд



HP -=
NetworkVar

FX +
effectRpc

PlayerShooting.cs

⑥ Стрельба

```
public class PlayerShooting : NetworkBehaviour {
    public NetworkObject projectilePrefab;
    public Transform muzzle;
    float lastShot; const float COOLDOWN = 0.5f;

    void Update() {
        if (!IsOwner) return;
        if (Input.GetKeyDown(KeyCode.Space) && Time.time > lastShot + COOLDOWN) {
            lastShot = Time.time;
            Vector3 dir = Camera.main.transform.forward;
            ShootServerRpc(dir);
        }
    }

    [ServerRpc(RequireOwnership = true)]
    void ShootServerRpc(Vector3 direction, ServerRpcParams p = default) {
        if (p.Receive.SenderClientId != OwnerClientId || direction.sqrMagnitude < 0.01f) return;
        var obj = Instantiate(projectilePrefab, muzzle.position, Quaternion.LookRotation(direction));
        obj.SpawnWithOwnership(OwnerClientId);
    }
}
```

Projectile.cs

⑥ Стрельба

```
public class Projectile : NetworkBehaviour {  
    public float speed = 20f, damage = 25f, lifetime = 3f;  
  
    void Update() {  
        if (!IsServer) return; // движение только на сервере  
        transform.position += transform.forward * speed * Time.deltaTime;  
        if (lifetime <= 0) GetComponent<NetworkObject>().Despawn(true);  
        lifetime -= Time.deltaTime;  
    }  
  
    void OnTriggerEnter(Collider other) {  
        if (!IsServer) return;  
        var hp = other.GetComponent<HealthComponent>();  
        if (hp != null) hp.TakeDamage(damage, OwnerClientId); // OwnerClientId = стрелок!  
        GetComponent<NetworkObject>().Despawn(true);  
    }  
}
```


Чек-лист перед Практикой 2

Что ты должен знать и уметь

Движение

- ☐ NetworkTransform на Player Prefab
- ☐ if (!IsOwner) return в Update
- ☐ Interpolation включён в Inspector

Ownership

- ☐ Инициализация только в OnNetworkSpawn
- ☐ Чистка подписок в OnNetworkDespawn
- ☐ Despawn(true) вместо Destroy

Камера

- ☐ Одна камера на сцене, следит через IsOwner
- ☐ Cursor.lockState только для IsOwner

Стрельба

- ☐ Снаряд спавнится только на сервере
- ☐ ShootServerRpc с 4 guard-клозами
- ☐ SpawnWithOwnership для начисления очков
- ☐ PlaySoundClientRpc — звук у всех